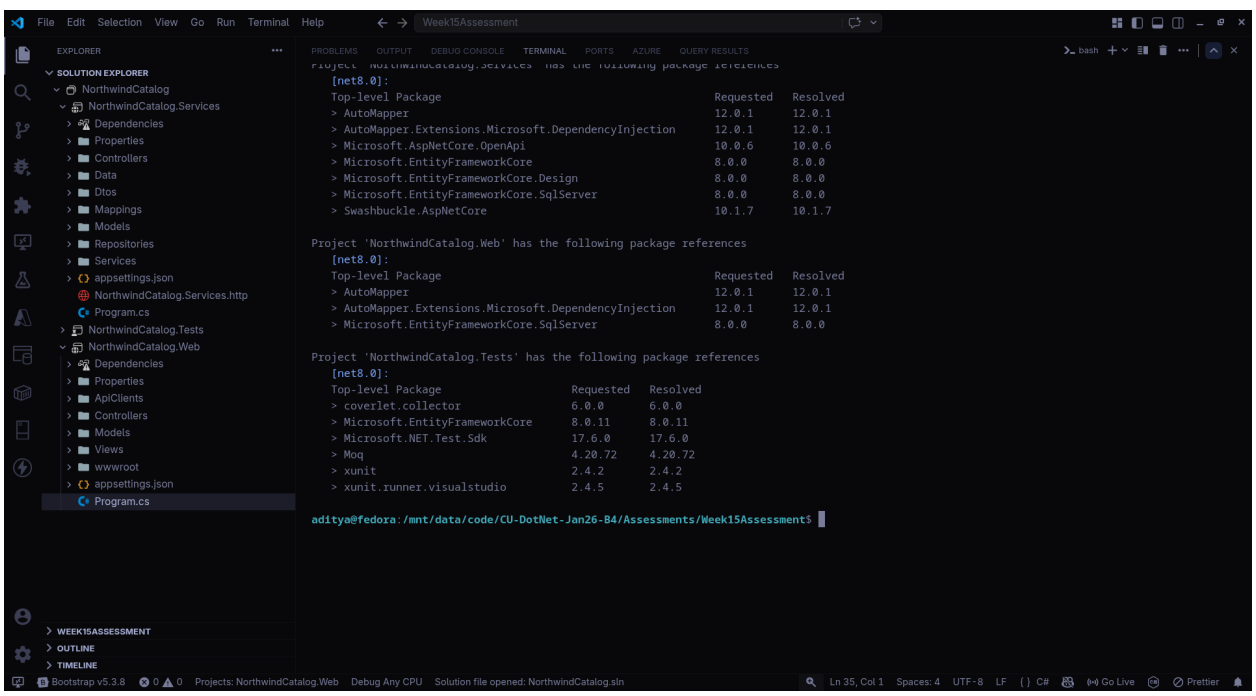
NORTHWINDCATALOG PROJECT - IMPLEMENTATION STEPS

STEP 1: CREATED PROJECT STRUCTURE

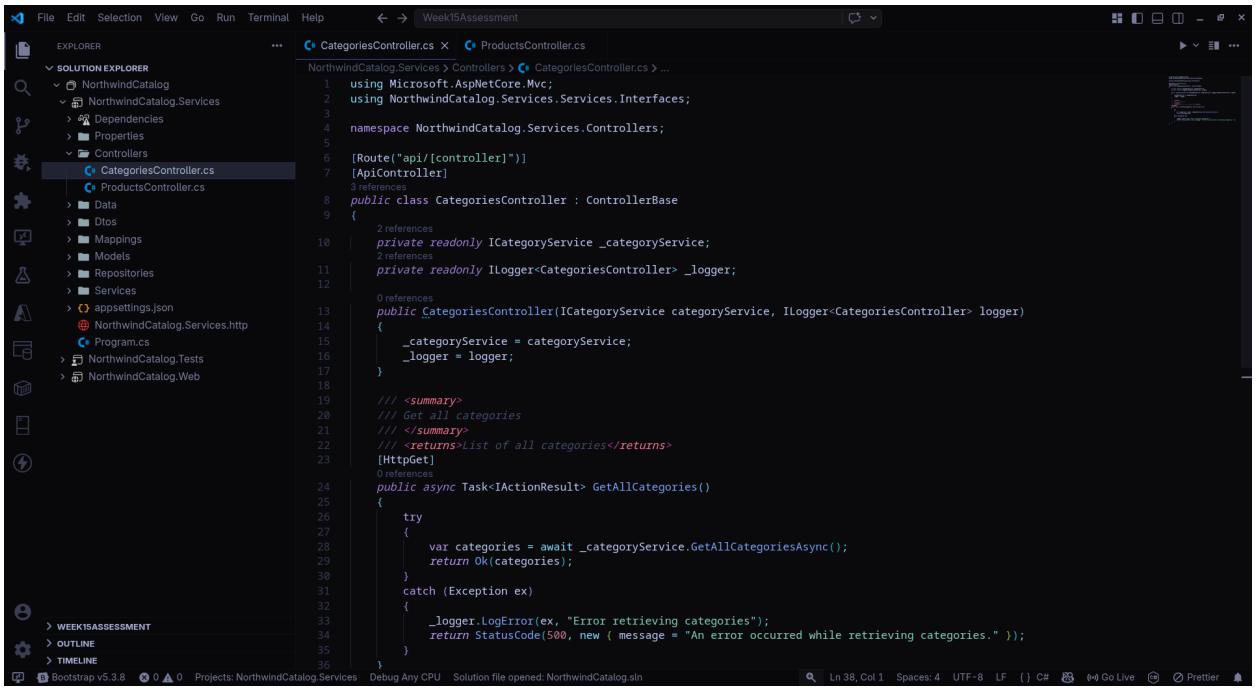
- Created NorthwindCatalog.Services (Web API project)
- Created NorthwindCatalog.Web (MVC project)
- Created NorthwindCatalog.Tests (xUnit project)
- Organized each project with Models, DTOs, Controllers, Services, Repositories

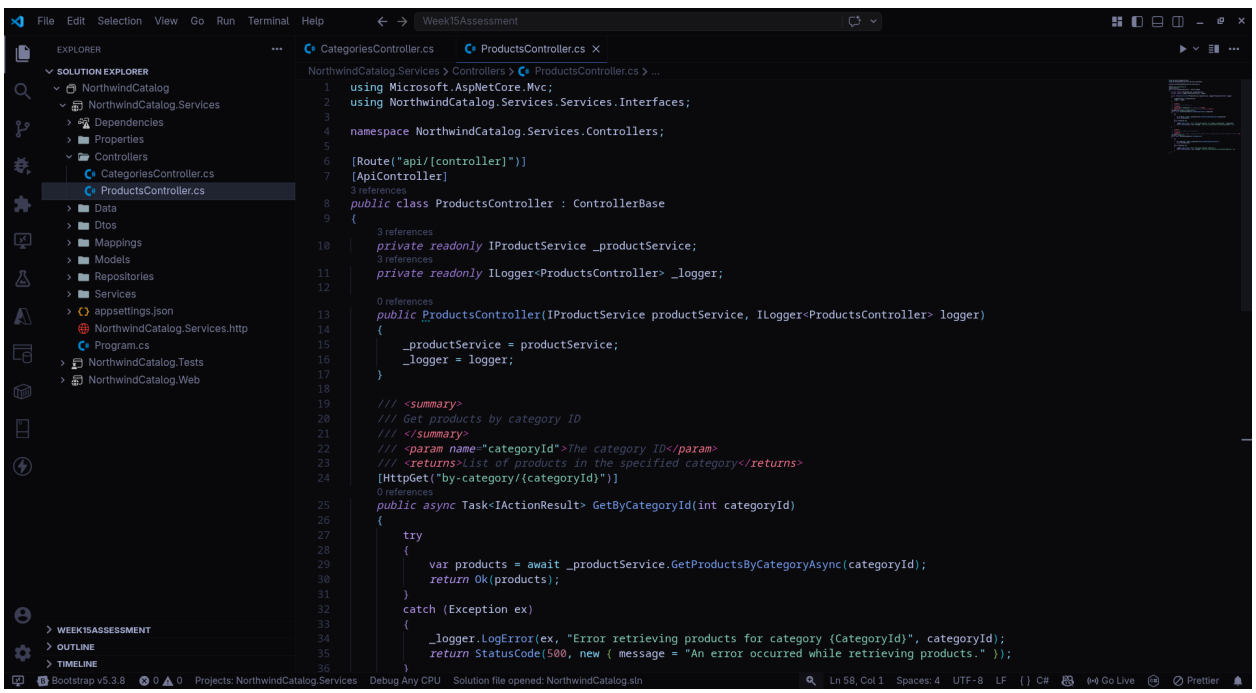
STEP 2: BUILD SERVICES API LAYER

- Installed NuGet packages: EntityFrameworkCore, AutoMapper, Swagger
- Created NorthwindContext (DbContext) for database access
- Created Entity Models: Category, Product
- Created Data Transfer Objects (DTOs): CategoryDto, ProductDto, CategorySummaryDto
- Created Repository pattern: IRepository interfaces and Repository implementations
- Created Service layer: IService interfaces and Service implementations



STEP 3: BUILD API CONTROLLERS

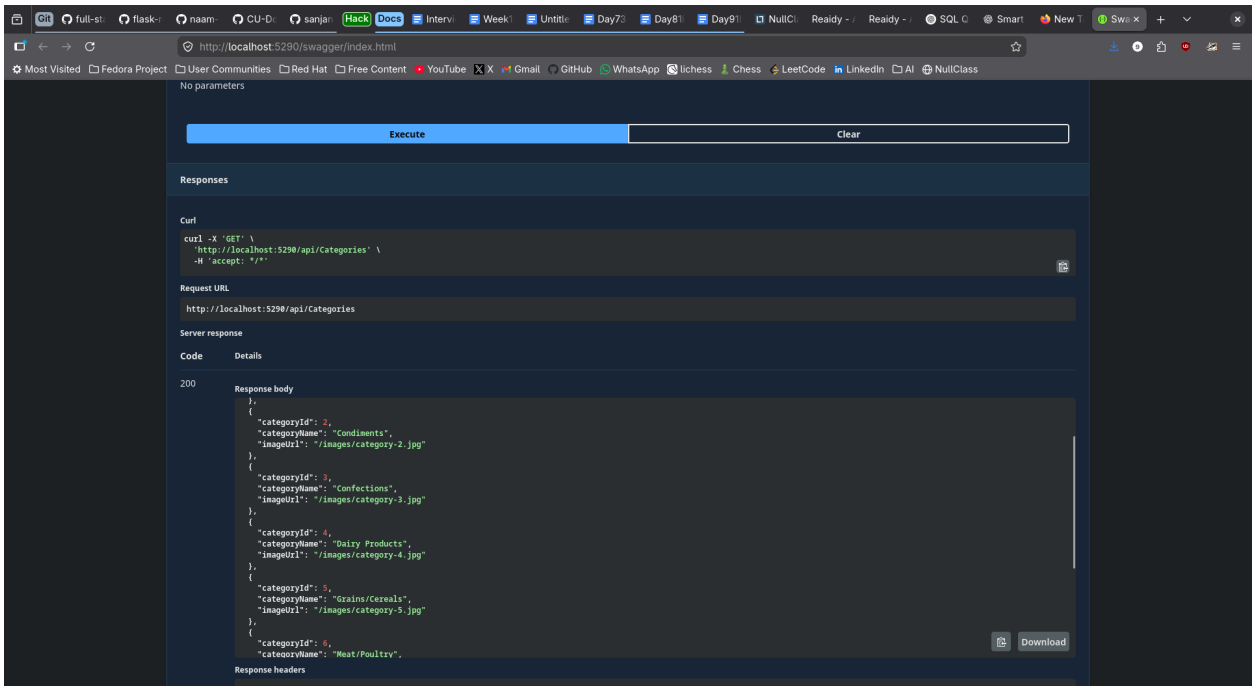




- CategoriesController:
  - GET /api/categories → Returns all categories

- ProductsController:
  - GET /api/products/by-category/{id} → Products filtered by category
  - GET /api/products/summary → Category analytics (count, avg price, etc.)

STEP 4: CONFIGURE API INFRASTRUCTURE



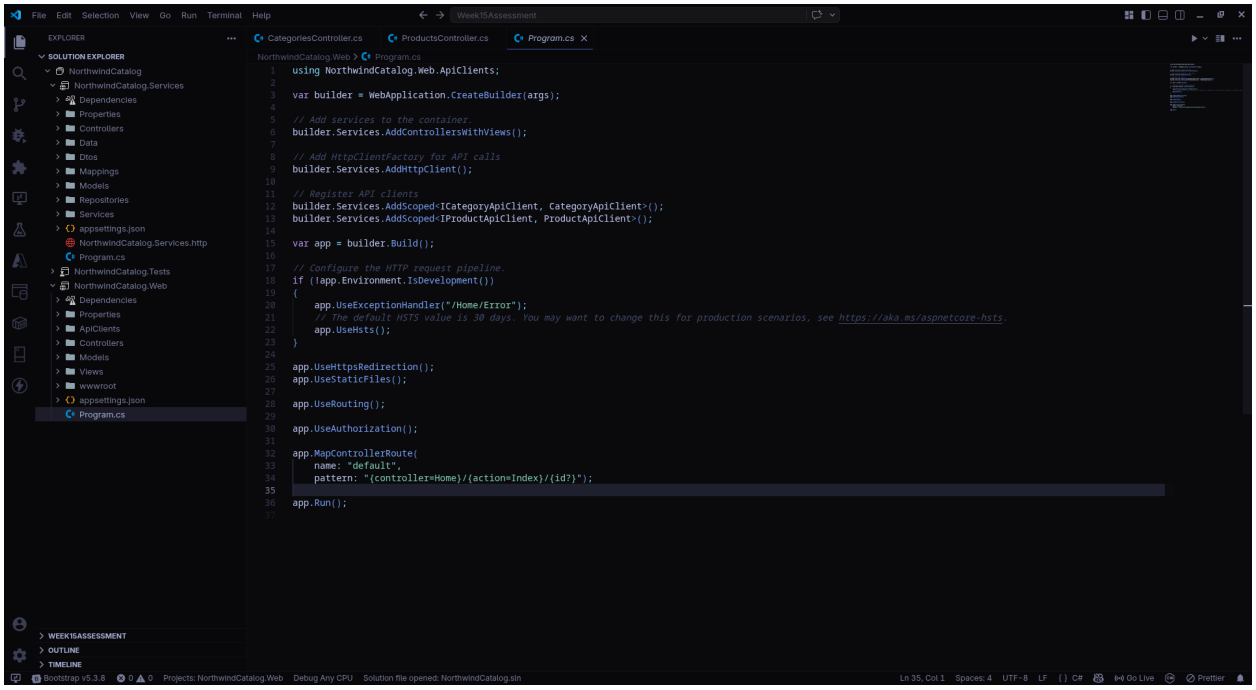
- Program.cs setup:
  - Register DbContext with SQL Server
  - Register Repositories (scoped lifetime)
  - Register Services (scoped lifetime)
  - Configure AutoMapper for Entity → DTO mapping
  - Enable Swagger/OpenAPI documentation
  - Configure CORS to allow Web project calls
  - Setup dependency injection

STEP 5: BUILD WEB MVC LAYER

- Removed local database access
- Created API Clients:
  - ICategoryApiClient interface
  - IProductApiClient interface
  - CategoryApiClient implementation (HTTP calls to /api/categories)
  - ProductApiClient implementation (HTTP calls to /api/products/*)

- Updated HomeController:
  - Inject API clients instead of services
  - Categories() action → calls GetAllCategoriesAsync()
  - Products() action → calls GetProductsByCategoryAsync()
  - Summary() action → calls GetCategorySummariesAsync()

STEP 6: CONFIGURE WEB PROJECT



- Program.cs setup:
  - Register MVC services
  - Register HttpClientFactory
  - Register API clients with dependency injection
  - Remove: DbContext, Repositories, Services (all in Services project now)

- appsettings.json:
  - Set ApiBaseUrl to point to Services API (https://localhost:7001)

STEP 7: CREATE VIEWS (MVC UI)

- Index.cshtml → Landing page with navigation
- Categories.cshtml → Display all categories (Bootstrap cards)
- Products.cshtml → Display products by category
- Summary.cshtml → Show analytics table

STEP 8: IMPLEMENT UNIT TESTS

- UnitTest1.cs → ProductDto InventoryValue calculations (4 tests)
- ProductServiceTests.cs → Service layer logic (3 tests)
- CategoryServiceTests.cs → Service layer and DTOs (4 tests)
- Used Moq for mocking repositories
- All 11 tests passing

ARCHITECTURE FLOW

User Request
↓
Web MVC Controller (HomeController)
↓
API Client (CategoryApiClient / ProductApiClient)
↓
HTTP Request to Services API
↓
Services API Controller (CategoriesController / ProductsController)
↓
Service Layer (Business Logic)
↓
Repository Layer (Database Access)
↓
Entity Framework Core / SQL Server
↓
[JSON Response back through the chain]
↓
Web Page Rendered (Razor View)